Lab 13 — Deployment Experiment Report / รายงานทดลอง deployment

Dr. Pisit Praiwattana, your mentor in 2026

Student ID / รหัสนักศึกษา: 6788185

Name / ชื่อ: Daranpob Sianghen

Laptop OS, VM OS and Docker versions / OS แล็ปท็อป, OS ของ VM และรุ่น Docker:
Windows + Docker Desktop (linux-desktop)

Repository URL / URL repository: https://github.com/boncedy/itcs258-l13-boncedy

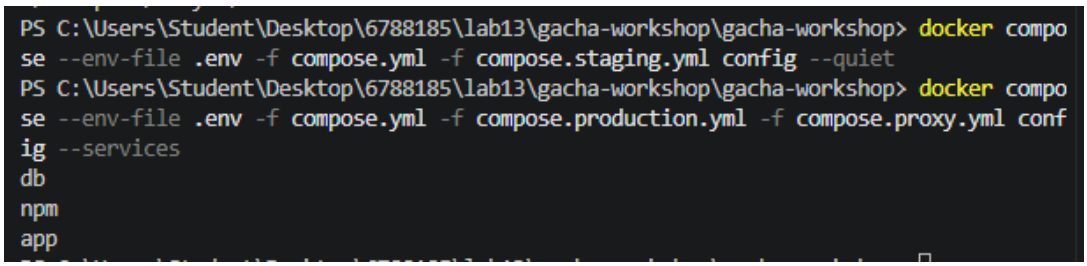
Use **one gacha project** for required experiments **01–04**. Explain in English or Thai. Every screenshot must come from your own actual screen, and passwords/tokens must be hidden.

ใช้ **gacha โปรเจกต์เดียว** ทำงานหลัก **01–04** อธิบายภาษาอังกฤษหรือไทยได้
ภาพต้องมาจากหน้าจอที่รันจริงของตนเอง และปิดรหัสผ่าน/token

01 — Compose selection and direct port / เลือก Compose และเข้าผ่านพอร์ตตรง

- Show sanitized base/local configuration and the complete -f command used.
- Explain .env interpolation versus explicit container variables.
- Show configuration validation, service status and http://localhost:8085/test.
- Show actual virtual recharge, exchange, draw and history results using your own ID.
- Explain how the staging/production preview differs; no three simultaneously running stacks are required.

Original local history ID, result and creation time / ID history ผล และเวลาสร้างในเครื่อง:

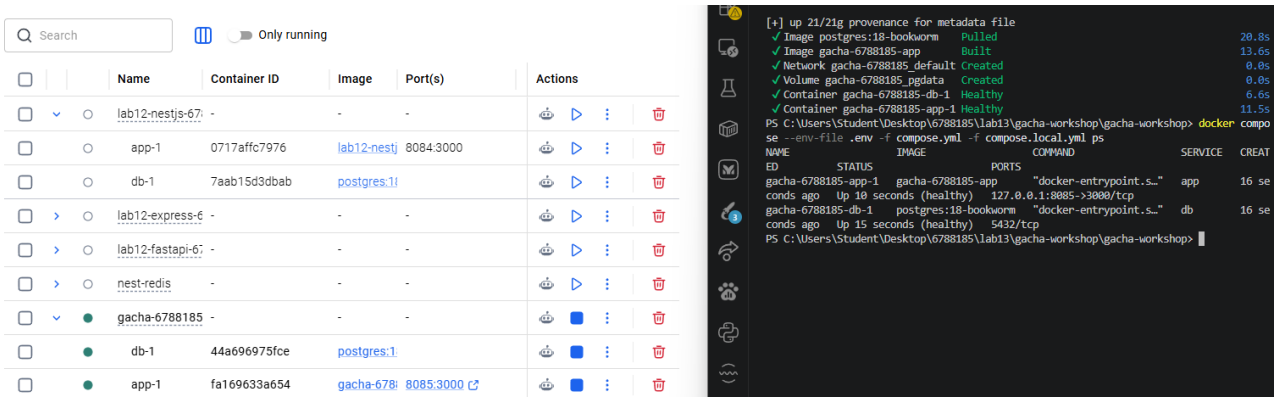


Evidence and explanation / หลักฐานและคำอธิบาย:

```
PS C:\Users\Student\Desktop\6788185\lab13\gacha-workshop\gacha-workshop> Copy-Item .e
nv.example .env
PS C:\Users\Student\Desktop\6788185\lab13\gacha-workshop\gacha-workshop>

PS C:\Users\Student\Desktop\6788185\lab13\gacha-workshop\gacha-workshop> docker compo
se --env-file .env -f compose.yml -f compose.local.yml config --quiet
PS C:\Users\Student\Desktop\6788185\lab13\gacha-workshop\gacha-workshop> docker compo
se --env-file .env -f compose.yml -f compose.local.yml up -d --build --wait
[+] up 16/16
 ✓ Image postgres:18-bookworm Pulled                                    20.8s
#1 [internal] load local bake definitions
#1 reading from stdin 603B done
#1 DONE 0.0s
```

Compose reads variables from .env.example and interpolates them into compose.yml before starting the containers. For example, values such as ${POSTGRES_DB}, ${POSTGRES_USER}, and ${POSTGRES_PASSWORD} are filled in from the environment file. These are interpolation variables at Compose time. In addition, the services declare explicit runtime variables inside the environment: block, such as PORT, PGHOST, PGDATABASE, and PGUSER. Those are the variables the container actually receives at runtime. In other words, .env supplies values to Compose, while environment: tells containers what to run with.



The staging and production shapes were previewed with config only. This is enough to compare how the overlays change ports, services, and networking without starting three active stacks. The point is to inspect the merged Compose result, not to leave multiple live deployments running at once. This avoids port conflicts and keeps the local experiment clean.



02 — Local Nginx Proxy Manager / Nginx Proxy Manager ในเครื่อง

- Show the selected production + proxy files and the app's unpublished host ports.
- Show the shared Docker network and NPM upstream **app:3000**.
- Explain why NPM's localhost means NPM itself.
- Show the /backend/ custom location/rewrite and successful **/backend/test** buttons.
- Read the original local history row after changing topology, with the same retained database volume and student/project identity.

Request path / เส้นทางคำขอ:

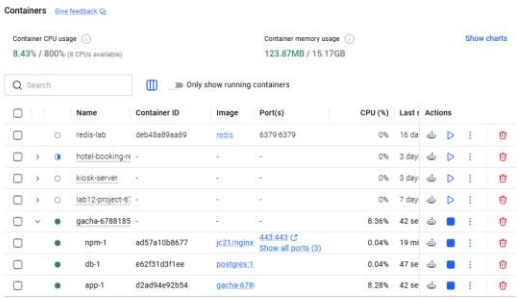
Laptop browser → NPM container (gacha.test:80, forward app:3000) → app container (:3000) → db container (db:5432)

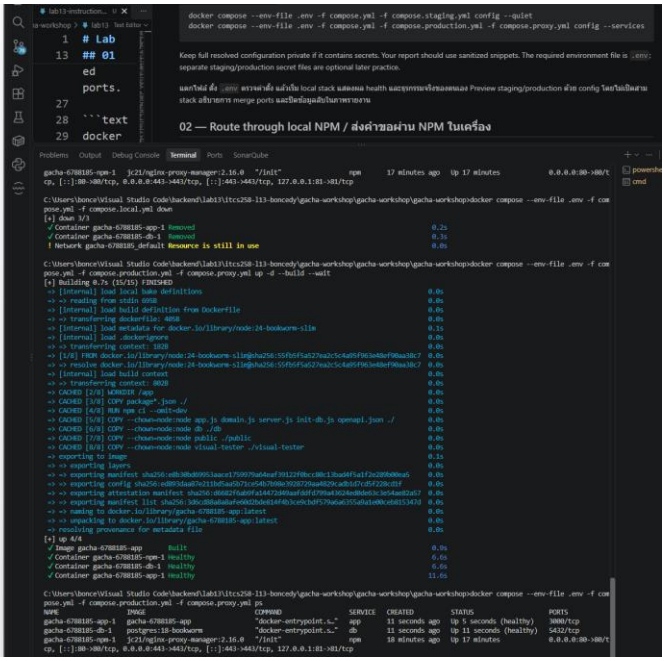
Item	Direct local	Local proxy
Browser URL	http://localhost:8085/test	http://gacha.test/backend/test
Host entry port	8085 published by compose.local.yml (127.0.0.1:8085 → app:3000)	80 published by the npm service (gacha.test resolves to 127.0.0.1); the app publishes no host port

Item	Direct local	Local proxy
App service/internal port	3000 (container port 3000, published as 8085 on the host)	3000, reached only as app:3000 inside gacha-6788185_default; no host port
Database volume name	gacha-6788185_pgdata	gacha-6788185_pgdata (same volume, reused — not recreated)
Original history ID + created_at	draw id 1 (transaction 3), Wind Fox / COMMON, 2026-11-05T13:37:28.394Z	unchanged: draw id 1, Wind Fox / COMMON, 2026-11-05T13:37:28.394Z

Evidence and explanation / หลักฐานและคำอธิบาย:

02 — Route through local NPM / ส่งคำขอผ่าน NPM ในเครื่อง





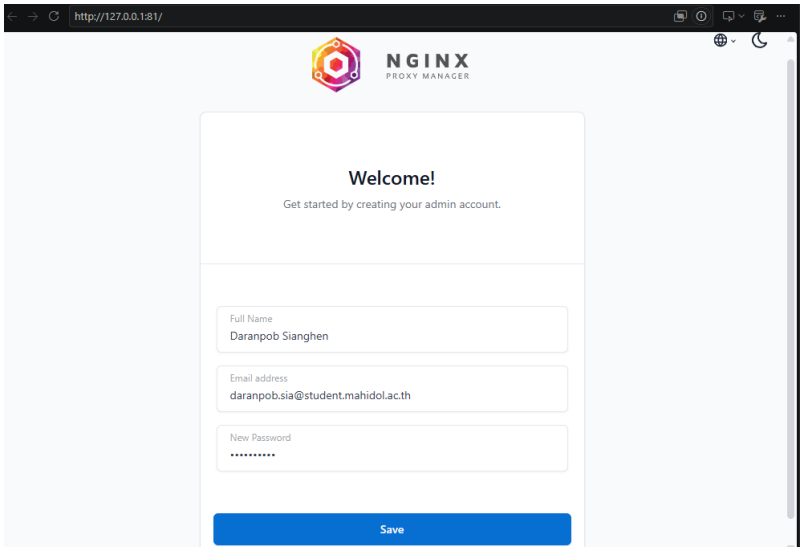
The proxy topology used three files only: compose.yml (base), compose.production.yml (adds restart: unless-stopped to app and db) and compose.proxy.yml (adds the npm service). compose.local.yml was left out deliberately, so the 127.0.0.1:8085:3000 mapping is gone: the app service in the base file declares no ports at all, and only NPM publishes host ports — 80:80, 443:443 and the admin UI bound to 127.0.0.1:81. The app and the database are therefore reachable only inside the project network, which is why docker compose ps lists npm as the single entry point while app shows no published port.

Task 02-1: Open NPM admin at http://127.0.0.1:81; complete the initial administrator setup shown by your version.

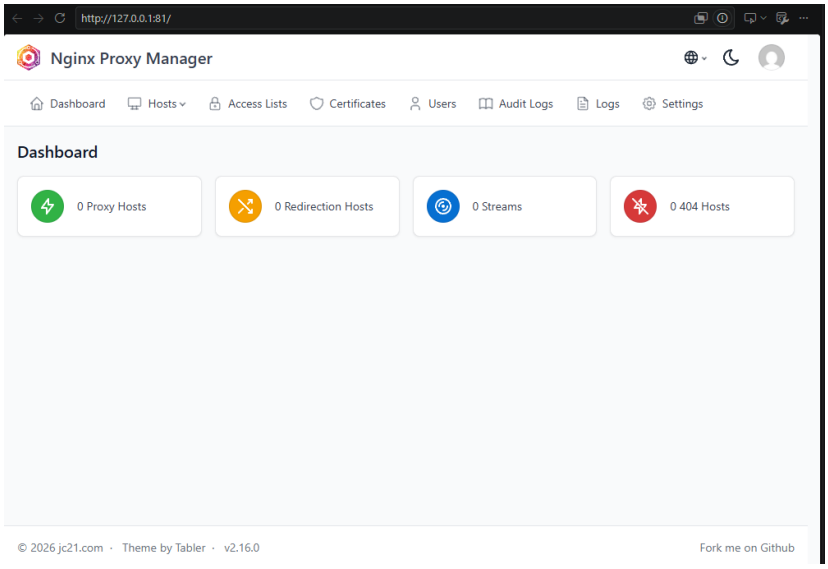
Initialize nginx:

```
PS C:\Users\bonce\Visual Studio Code\backend\lab13\itcs258-113-boncedy\gacha-workshop\gacha-workshop> docker compose --env-file .env -f compose.yml -f
 compose.production.yml -f compose.proxy.yml ps
NAME                 IMAGE                              COMMAND                  SERVICE   CREATED          STATUS                    PORTS
gacha-6788185-app-1  gacha-6788185-app                  "docker-entrypoint.s…"   app       11 minutes ago   Up 11 minutes (healthy)   3000/tcp
gacha-6788185-db-1   postgres:18-bookworm               "docker-entrypoint.s…"   db        11 minutes ago   Up 11 minutes (healthy)   5432/tcp
gacha-6788185-npm-1  jc21/nginx-proxy-manager:2.16.0    "/init"                  npm       29 minutes ago   Up 29 minutes             0.0.0.0:80->80/t
cp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 127.0.0.1:81->81/tcp
PS C:\Users\bonce\Visual Studio Code\backend\lab13\itcs258-113-boncedy\gacha-workshop\gacha-workshop> docker network inspect gacha-6788185_default --f
ormat '{{range .Containers}}{{.Name}} {{end}}'
gacha-6788185-npm-1 gacha-6788185-app-1 gacha-6788185-db-1
```

Before login:



After:

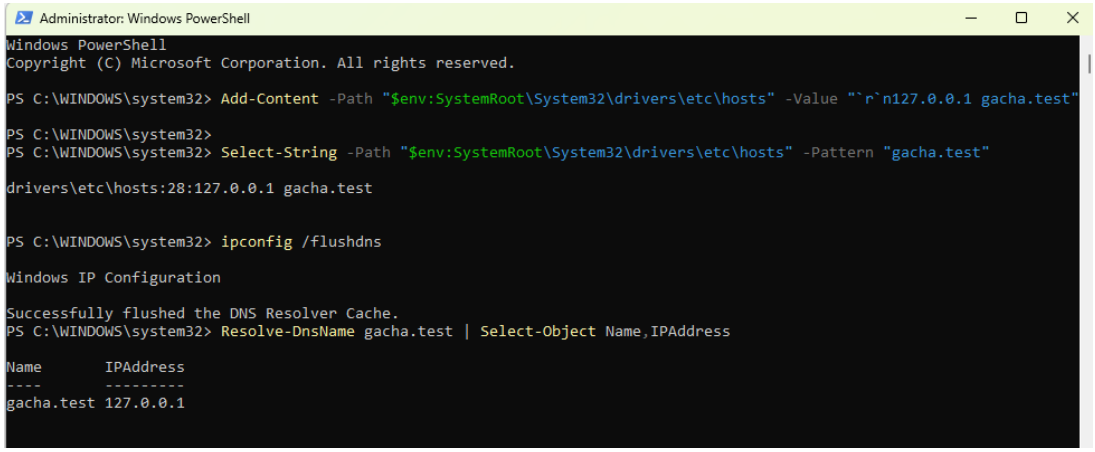


app and npm join the same project default network, gacha-6788185_default, where Compose registers each service name as a DNS alias. That shared network is why the NPM proxy host can use the upstream app:3000 — the service name and the internal port, never a host port. Inside the npm container, localhost (or 127.0.0.1) means the NPM container itself: every container has its own loopback interface, so localhost:3000 from NPM asks for a process inside the NPM image, where nothing listens on 3000, and the

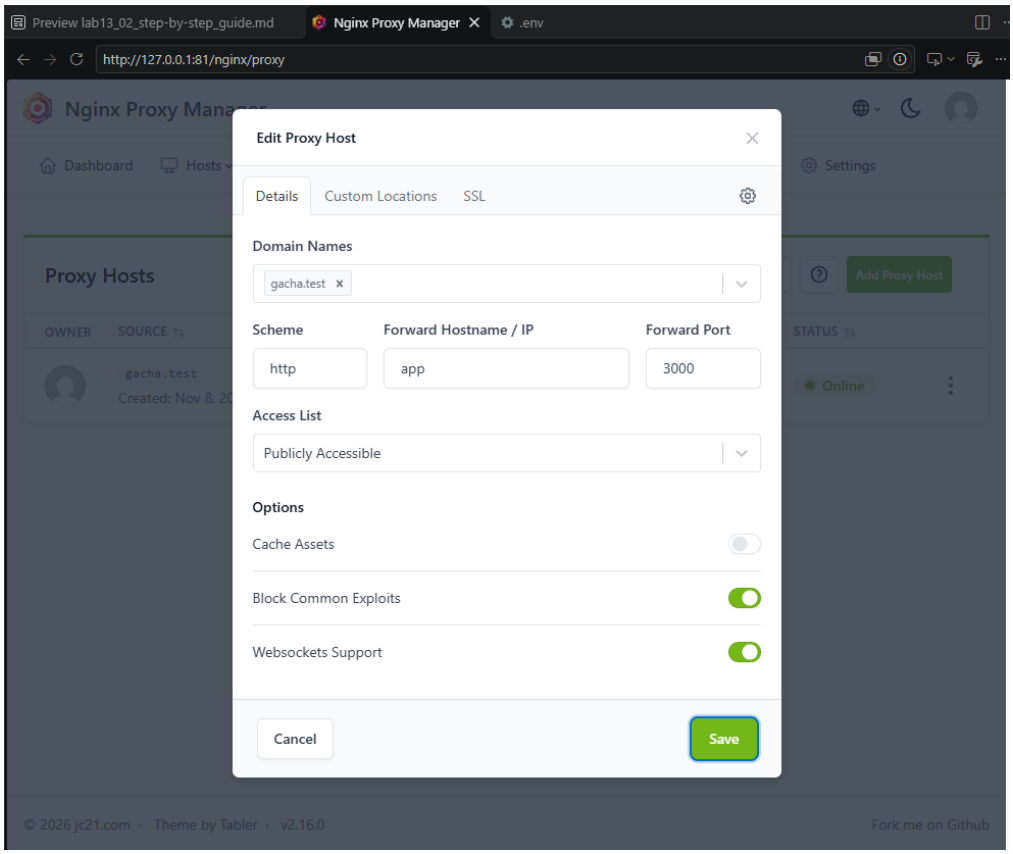
proxy would answer 502. Docker's embedded DNS resolves app to the app container's IP on the shared network instead.

Task 02-2: Add 127.0.0.1 gacha.test to your laptop's hosts file, preserving existing entries.

Adding host:

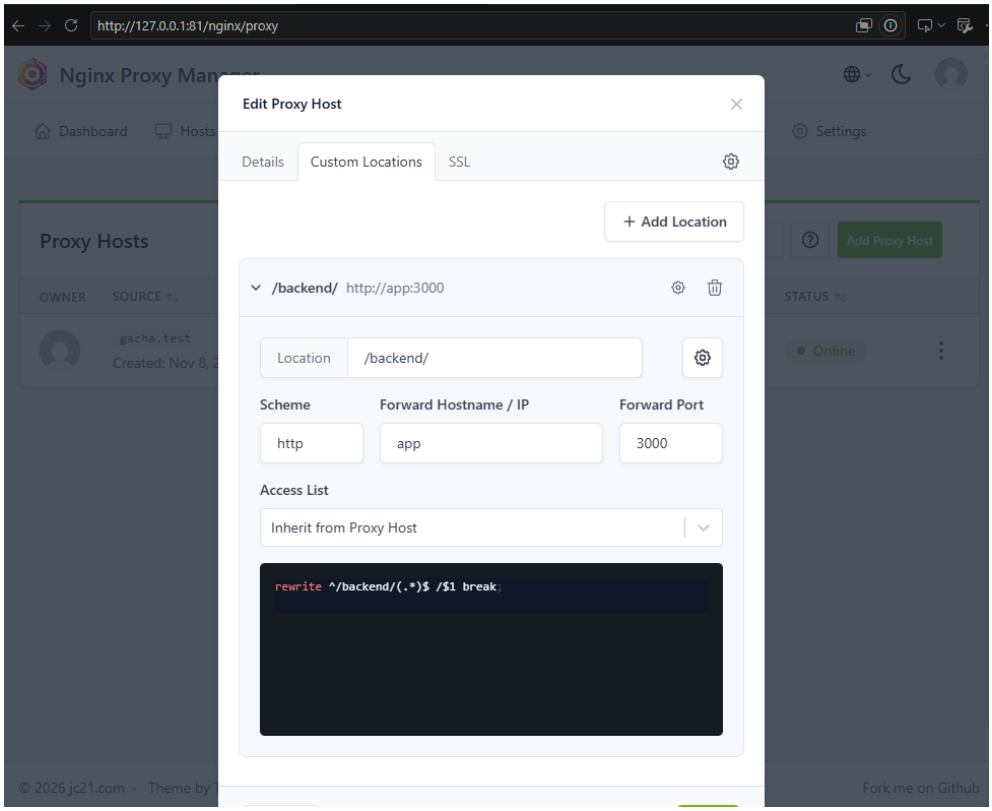


Task 02-3: Create a Proxy Host for gacha.test, scheme http, forward hostname app, port 3000. Use local HTTP for this .test name.



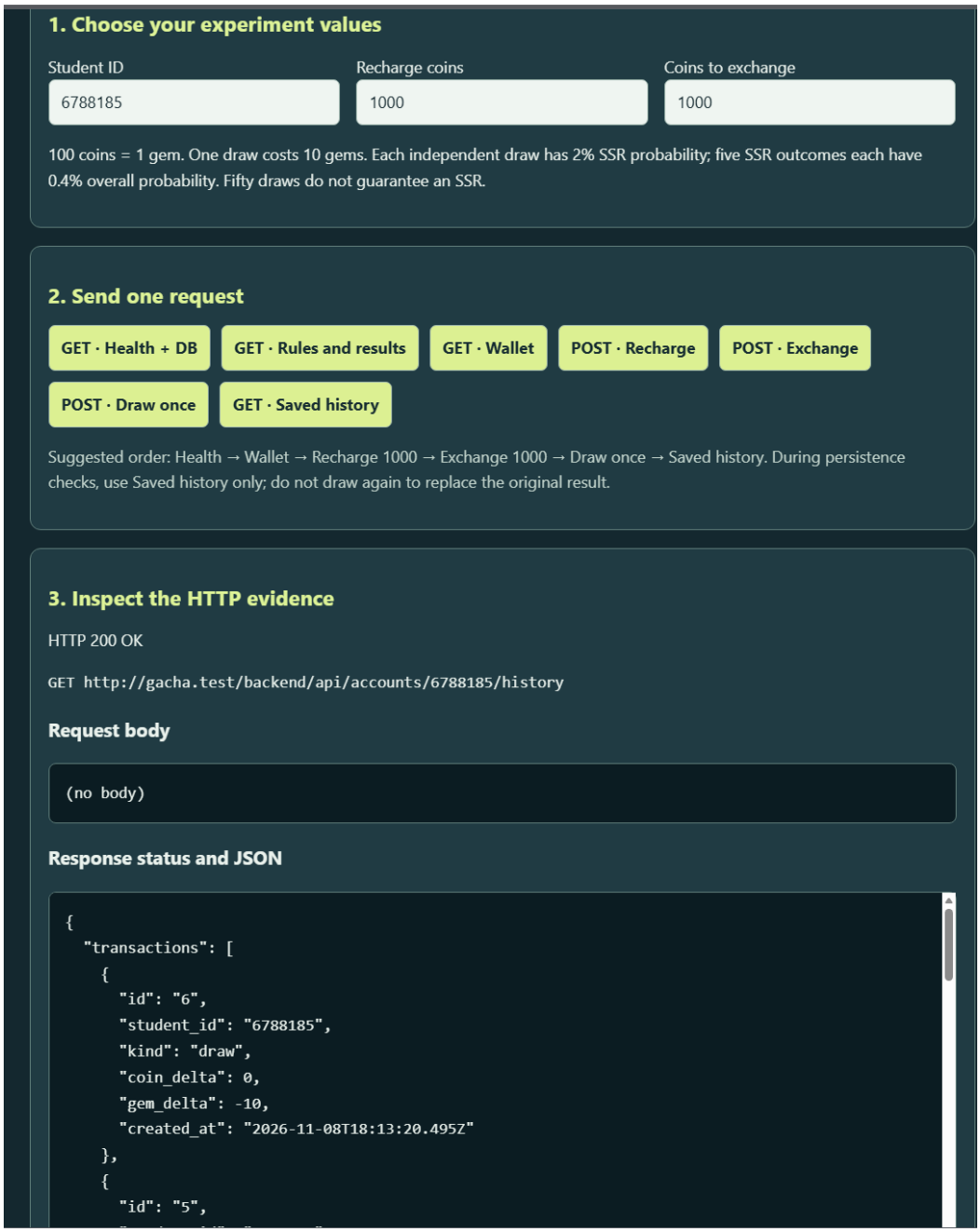
Task 02-4: Add a Custom Location /backend/, also forwarding to http, app, 3000. In that location's gear/advanced field put only:

rewrite ^/backend/(.*)$ /$1 break;



The Proxy Host for gacha.test forwards http to app:3000, and its Custom Location /backend/ forwards to the same upstream with only the advanced rewrite rewrite ^/backend/(.*)$ /$1 break;. NPM generates the surrounding location block and proxy_pass itself, so pasting another location or a second proxy_pass would duplicate them. Because the prefix is stripped, /backend/test reaches the app as /test, and the test page derives its API prefix from its own URL, so every button calls /backend/api/... and is rewritten to /api/... . Health, wallet, recharge, exchange, draw and history all answered 200/201 through the proxy, so the logic buttons work behind the prefix as well.

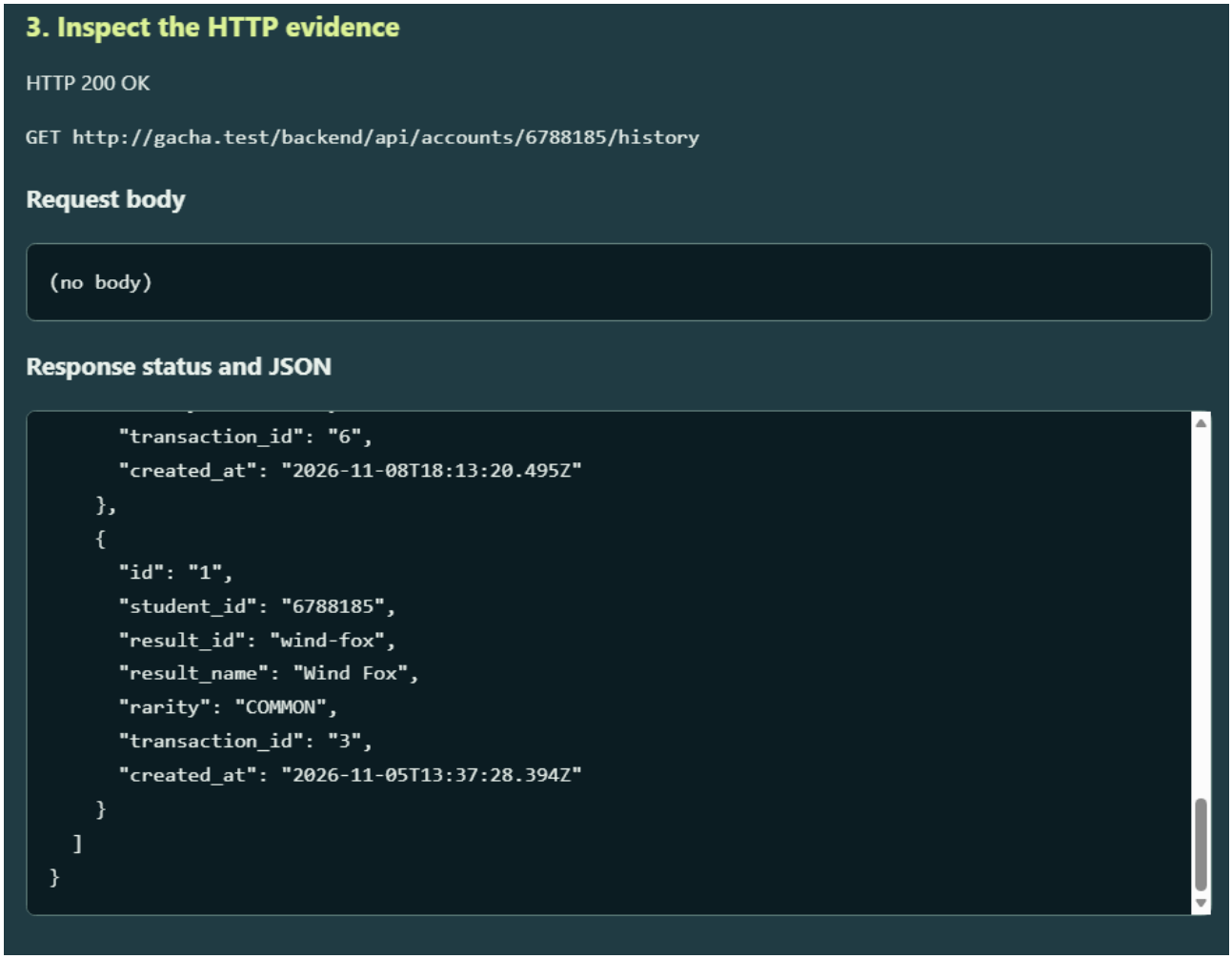
Task 02-5: Save and open http://gacha.test/backend/test. Check that health and all logic buttons work. The proxy sends /backend/test to the app as /test; its API buttons must also pass through /backend/.



The local stack was stopped with docker compose down, without -v, so Docker removed the containers and the project network but kept the named volume. STUDENT_ID stayed 6788185, so the project name gacha-6788185 — and with it the volume gacha-6788185_pgdata — was unchanged when the production + proxy stack started. The history row that this volume already held was read back through the proxy, unchanged and never re-inserted: draw id 1, Wind Fox / COMMON, created_at 2026-11-05T13:37:28.394Z. The button test in step 8 added new transactions and draw id 2 (River Sprite), but it did not replace or modify row 1. Only the app container and the request path changed; the database storage did not.

Task 02-6: Retrieve the history row saved in experiment 01 without drawing a replacement. Confirm the database volume identity is unchanged.

The saved history still returns the original row: id 1, Wind Fox, created_at 2026-11-05T13:37:28.394Z — read back through the proxy, not re-inserted. The River Sprite draw created by the button test sits above it as a separate newer row (id 2), and no stored row was replaced. The database volume gacha-6788185_pgdata is the same one the stack started with.



Task 02-7: Draw the path: laptop browser → NPM container → app:3000 → db:5432. Explain why localhost inside NPM means NPM itself, not the separate app container.

Because each container has its own loopback, that means 127.0.0.1 never leaves its container. Inside NPM container, nothing listens to :3000, so the :3000 just points to NPM itself.

03 — Ubuntu VM / VPS model

- Identify the Ubuntu guest address and whether access uses bridged networking or NAT/forwarded ports.
- Show the basic shell operations, Docker Engine service, Compose startup, status and relevant logs.
- Show SSH-tunnel access to loopback-only NPM admin and browser access to the VM's /backend/test route.
- Explain that VM data storage is separate from laptop storage.
- Create a VM record, redeploy/recreate the app, and retrieve the same original VM record.
- Explain the public-VPS requirements: actual DNS, provider/host network rules, deliberate public ports, SSH access and HTTPS certificate.

VM network diagram / แผนภาพเครือข่าย VM:

Item	Before app replacement	After app replacement
App container ID		
DB volume identity		
Original VM history ID + created_at		

Evidence and explanation / หลักฐานและคำอธิบาย:

04 — Railway deployment / Deploy บน Railway

- Show the app source/root directory and Dockerfile build evidence.
- Show app + PostgreSQL service roles and sanitized variable **names/references**, not literal secrets.
- Show HTTPS /test, database-aware /health and successful virtual transactions.
- Record one online history row before redeploying the app and retrieve it afterward without reinserting/recreating it.
- Explain where the persistent database storage lives, what backup/recovery options you inspected, and why laptop rows do not appear automatically.

Public app URL / URL แอปสาธารณะ:

Item	Before app redeploy	After app redeploy
App deployment identifier		
Original online history ID		
Result and creation time		

Evidence and explanation / หลักฐานและคำอธิบาย:

Diagnosis and reflection / วิเคราะห์ปัญหาและสะท้อนผล

Symptom → hypothesis → check → fix / อาการ → สมมติฐาน → การตรวจ → วิธีแก้:

What changed between a VM and Railway? / สิ่งใดเปลี่ยนระหว่าง VM กับ Railway?

What operational work still belongs to the developer? / ผู้พัฒนายังต้องดูแลงานระบบอะไร?

Self-quiz topic to revisit / หัวข้อจาก self-quiz ที่ต้องทบทวน:

In-room check and final submission / ตรวจในห้องและส่งงาน

- ☐ I completed 01–04 using the same working example.
- ☐ I demonstrated my actual experiment to the LA in the room.
- ☒ All evidence is from my own screen and readable.
- ☒ My report includes request paths, real results, persistence comparisons and repository URL.
- ☒ No actual passwords or tokens are visible.
- ☒ The single PDF is named **student_ID_lab13.PDF**, using my actual ID.
- ☒ I uploaded it to MyCourse by its deadline and opened the uploaded PDF to verify it.

Optional extension: another backend may be described here, but it is not required.

แบบฝึกเสริม: อธิบายแบ็กเอนด์อื่นเพิ่มเติมได้ แต่ไม่ใช่งานบังคับ